Reflection Report

Working through this assignment provided a solid hands-on understanding of filesystem operations and kernel data structures in xv6. The most subtle challenge in Task 2 was handling directory entry traversal correctly without leaking file descriptors or triggering string buffer overflows. Because directory entry names in xv6 can take up all 14 bytes without a null terminator, directly using standard string operations on de.name caused intermittent buffer reads. Resolving this required explicit memory copying with memmove and manual null termination. Additionally, managing open file descriptors during recursive directory traversal in du required strict discipline, as failing to close directory handles before recursing deeper quickly exhausted the kernel file table limit.

Task 3 introduced architectural constraints regarding disk layout and kernel state synchronization. Modifying struct dinode required careful attention to struct alignment because on-disk inodes must remain exactly 64 bytes (a power of two). Decreasing NDIRECT from 12 to 11 successfully freed up space for the 32-bit timestamp field while preserving the overall struct size. Beyond structural modifications, tracing every kernel execution path where files are allocated, truncated, or modified to ensure timestamp updates was an insightful exercise. It clearly demonstrated how in-memory inode structures interact with on-disk representations through iupdate and ilock.

Overall, this assignment was a very effective exercise in systems programming and operating system design. Implementing mv through link and unlink system calls demystified how UNIX filesystems perform atomic renames without copying data on disk. Implementing recursive traversal for du and adding inode metadata reinforced how user-space programs interact with kernel abstractions via system calls like fstat. The tasks were well-structured and provided a rewarding dive into low-level C programming.